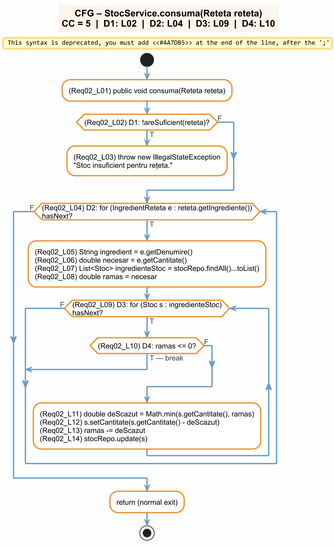
This editor runs entirely in your browser thanks to our JavaScript rendering engine. Edit the code on the left and see the diagram update instantly!

```
@startuml CFG_StocService_consuma

skinparam backgroundColor #FDF4EC
skinparam ActivityBackgroundColor white
skinparam ActivityBorderColor #E8983A
skinparam ActivityBorderThickness 2
skinparam ActivityDiamondBackgroundColor white
skinparam ActivityDiamondBorderColor #E8983A
skinparam ArrowColor #6B9EC7
skinparam ArrowFontColor #444444
skinparam ActivityFontSize 11

title **CFG – StocService.consuma(Reteta reteta)**\nCC =
5 | D1: L02 | D2: L04 | D3: L09 | D4: L10

start

:(Req02_L01) public void consuma(Reteta reteta);

if ((Req02_L02) D1: !areSuficient(reteta)?) then (T)
  #4A7DB5:(Req02_L03) throw new
IllegalStateException\n"Stoc insuficient pentru reţeta.";
  kill
else (F)
endif

while ((Req02_L04) D2: for (IngredientReteta e :
reteta.getIngrediente()))\nhasNext?) is (T)

  :(Req02_L05) String ingredient = e.getDenumire()
(Req02_L06) double necesar = e.getCantitate()
(Req02_L07) List<Stoc> ingredienteStoc =
stocRepo.findAll()...toList()
(Req02_L08) double ramas = necesar;

  while ((Req02_L09) D3: for (Stoc s :
ingredienteStoc)\nhasNext?) is (T)

    if ((Req02_L10) D4: ramas <= 0?) then (T — break)
      break
    else (F)
    endif

    :(Req02_L11) double deScazut =
Math.min(s.getCantitate(), ramas)
```



PlantUML at a Glance

Getting Started

PlantUML is a highly versatile tool that facilitates the rapid and straightforward creation of a wide array of diagrams.

Utilizing a simple and intuitive language, users can effortlessly draft various types of diagrams. For a detailed exploration of the language's capabilities and syntax, please refer to the PlantUML Language Reference Guide.

If you are new to PlantUML, we recommend starting with our quick start page to get up and running quickly. Should you have any questions, our F.A.Q. page is a valuable resource. Additionally, PlantUML can be seamlessly integrated with a variety of other tools to enhance your workflow.

Supported UML Diagrams

With PlantUML, you can create well-structured UML diagrams, including but not limited to:

- Sequence diagram
- Usecase diagram
- Class diagram
- Object diagram
- Activity diagram *(Find the legacy syntax here)*
- Component diagram
- Deployment diagram
- State diagram
- Timing diagram

Supported Non-UML Diagrams

Beyond the traditional UML diagrams, PlantUML also supports the creation of various other diagram types, such as:

- JSON data
- YAML data
- EBNF diagram
- Regex diagram
- Network diagram (nwdiag)
- UI mockups (salt)
- Archimate diagram
- Specification and Description Language (SDL)
- Ditaa diagram
- Gantt diagram
- Chronology diagram
- MindMap diagram
- WBS diagram
- Mathematics with AsciiMath or JLaTeXMath notation
- Information Engineering (IE) diagram
- Entity Relationship (ER) diagram
- Chart diagram
- Files diagram

Additional Features

Enhance the dynamism and informativeness of your diagrams with these additional features:

- Hyperlinks and tooltips to provide extra context and interactivity
- Rich text formatting, emoticons, Unicode, and icons with Creole for a visually appealing presentation
- OpenIconic icons for enhanced visual representation
- Sprite icons to add custom symbols
- AsciiMath mathematical expressions for precise mathematical representation

Input Formats

PlantUML allows you to generate diagrams from various source input formats:

- Source input data: *How and where diagrams can be written*

You can choose from different internal encodings:

- PlantUML Text Encoding

Layout Engines and Options

PlantUML allows you to use several different layout engines:

- Graphviz is the default engine. Relies on an external program;
- Smetana *(Use `!pragma Layout smetana` in the file or `-PLayout=smetana` on the command-line.)* is a internal port of Graphviz to Java. Tends to make slightly straighter arrows;
- VizJs uses JavaScript. *(Use `-graphvizdot vizjs` on the command line.)* Tends to space out nodes more, resulting in a bigger diagram;
- ELK (Eclipse Layout Kernel). *(Use `!pragma Layout elk` in the file or `-PLayout=elk` on the command-line.)* Supports only orthogonal layout, and doesn't cover all features.

Sequence diagrams have 2 engines:

- *Puma* is the older and still default engine;
- Teoz *(Use `!pragma teoz true` in the file or `-Pteoz=true` on the command-line)* is a new engine that has some unique features: *anchors (named arrows/messages), duration, nested boxes...*

  Achieving good layout with PlantUML is sometimes non-trivial. *See PlantUML GraphViz Layout by Mark George at University of Otago for more advice.*

Output Formats

Export your diagrams in a variety of formats to suit your needs, including:

- PNG for easy image sharing
- SVG for scalable vector graphics
- LaTeX for high-quality typesetting
- EPS (Encapsulated PostScript) for used with LaTeX
- ASCII art *(available only for sequence diagrams)* for a text-based representation

Enjoy

Get started today and elevate your diagramming capabilities with **PlantUML**. Transform your ideas into clear and professional diagrams effortlessly !